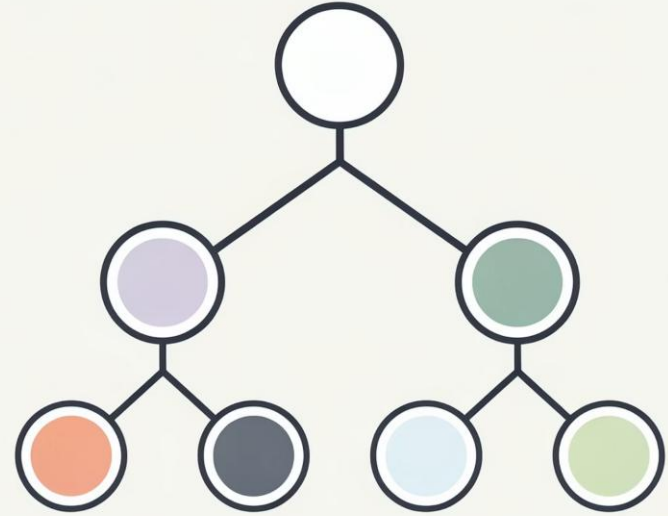


Árboles Binarios de Búsqueda

Organizando Datos de Forma Inteligente



¿Cómo buscarías un número en 10,000 elementos?

💡 Pregunta para la clase

Imagina que tienes una lista con **10,000 números** almacenados de forma aleatoria. Necesitas encontrar el número **7,842**. ¿Cuál sería tu estrategia?

- Reviso uno por uno desde el inicio
- Divido la lista a la mitad repetidamente
- Uso una estructura especial de datos

¿Por qué importa la estrategia?

En el peor caso, buscar uno a uno requiere **10,000 comparaciones**. Una estructura bien diseñada puede reducirlo a tan solo **14 comparaciones**. Eso es exactamente lo que logra un **Árbol Binario de Búsqueda**.



Nota docente: Activa una encuesta en Mentimeter o el chat del aula virtual. Pide a los estudiantes votar en tiempo real y conecta los resultados con la necesidad de estructuras eficientes.

¿Qué es un Árbol Binario de Búsqueda?

Un **BST (Binary Search Tree)** es una estructura de datos jerárquica donde cada nodo almacena un valor y mantiene una relación de orden estricta con sus hijos. Es la base de muchos algoritmos eficientes en ciencias de la computación.



Nodo

Unidad básica que almacena un valor y punteros a sus hijos izquierdo y derecho.



Raíz

El nodo superior del árbol. Es el punto de entrada para todas las operaciones.



Hojas

Nodos sin hijos. Representan los extremos del árbol.



Regla BST

Izquierda: valores menores. **Derecha:** valores mayores. Siempre.



¿Por qué usar un BST?

Búsqueda Secuencial (Lista)

Complejidad: $O(n)$

En el peor caso revisa **todos** los elementos. Para 10,000 datos → hasta 10,000 comparaciones.

Búsqueda en BST

Complejidad: $O(\log n)$

En cada paso **descarta la mitad** del árbol. Para 10,000 datos → máximo ~14 comparaciones.

Ventajas Clave del BST

01

Organización jerárquica natural

Los datos se ordenan automáticamente al insertarlos.

02

Búsquedas ultrarrápidas

Ideal para conjuntos de datos grandes y dinámicos.

03

Inserción y eliminación eficientes

Sin necesidad de reordenar toda la estructura.



 **Caso real:** Los motores de búsqueda usan árboles para indexar millones de páginas web en milisegundos.

Inserción en un BST

Para insertar un valor, el algoritmo **compara repetidamente** con cada nodo: si el valor es menor, va a la izquierda; si es mayor, va a la derecha. El proceso continúa hasta encontrar un espacio vacío.



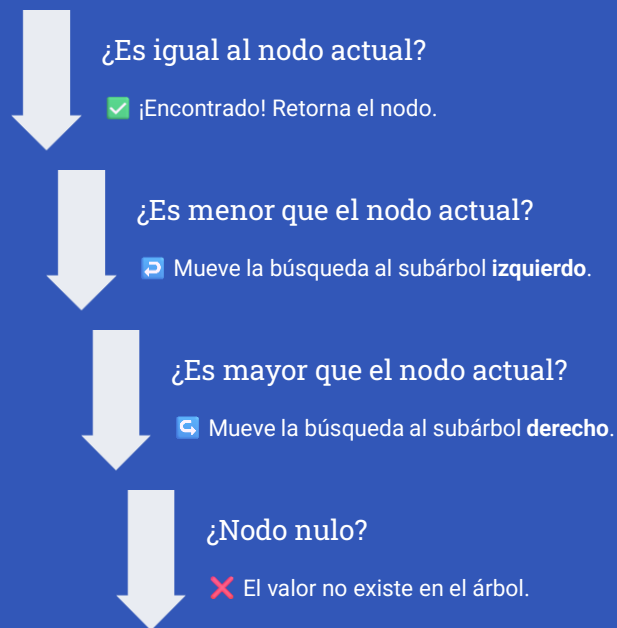
Código C++

```
Nodo* insertar(Nodo* nodo, int valor) {  
    if (nodo == nullptr)  
        return new Nodo(valor);  
    if (valor < nodo->dato)  
        nodo->izquierdo = insertar(  
            nodo->izquierdo, valor);  
    else if (valor > nodo->dato)  
        nodo->derecho = insertar(  
            nodo->derecho, valor);  
    return nodo;  
}
```

 **Pregunta interactiva:** Dado un árbol con raíz 50, hijos 30 y 70 — ¿dónde se insertaría el valor **45**?
Escríbelo en el chat.

Búsqueda en un BST

Flujo de Decisiones



Código C++

```
Nodo* buscar(Nodo* nodo, int valor) {  
    if (nodo == nullptr)  
        return nullptr; // No encontrado  
    if (valor == nodo->dato)  
        return nodo;    // Encontrado  
    if (valor < nodo->dato)  
        return buscar(  
            nodo->izquierdo, valor);  
    return buscar(  
        nodo->derecho, valor);  
}
```

📁 🏆 **Mini reto:** En un BST con raíz 50, busca el valor 35. ¿Cuántas comparaciones necesitas? Cuenta los pasos y responde en el chat.

Cada comparación **elimina la mitad** del árbol restante. Esto hace que la búsqueda sea **$O(\log n)$** en promedio.

Eliminación en un BST

La eliminación es la operación más compleja. Existen **tres escenarios** posibles según la cantidad de hijos que tenga el nodo a eliminar. Cada caso tiene su propia estrategia para mantener la propiedad del BST.



Caso 1: Nodo Hoja

El nodo **no tiene hijos**. Se elimina directamente sin afectar el resto del árbol. Es el caso más sencillo.



Caso 2: Un Solo Hijo

El nodo tiene **un hijo**. Se elimina el nodo y se **conecta directamente** su hijo con el padre del nodo eliminado.



Caso 3: Dos Hijos

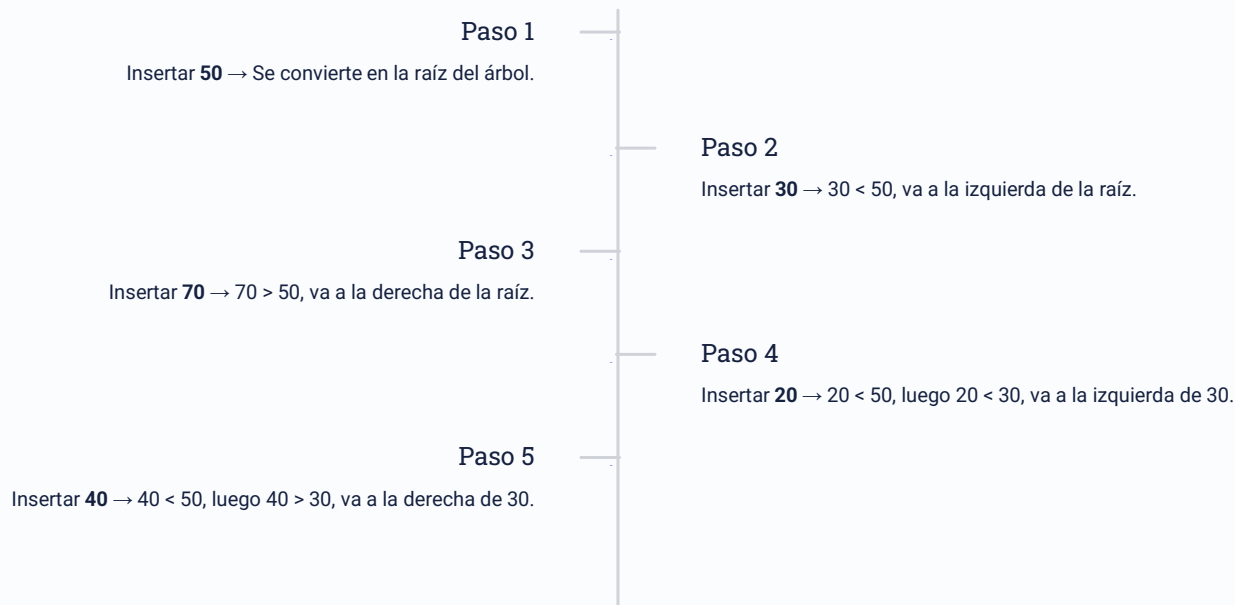
El nodo tiene **dos hijos**. Se busca el **sucesor inorden** (el menor del subárbol derecho) y se usa para reemplazar el nodo eliminado.



Actividad: Si el árbol tiene raíz 50, hijos 30 y 70 — ¿qué ocurre exactamente si eliminamos el nodo 50? ¿Cuál sería el nuevo nodo raíz? Debata en el chat.

Laboratorio: Construyamos un BST Juntos

Insertaremos los siguientes valores en orden: **50** → **30** → **70** → **20** → **40** → **60** → **80**. Predice la posición de cada nuevo nodo antes de verlo insertado.



✓ **Retroalimentación inmediata:** El árbol resultante es perfectamente balanceado. El recorrido Inorden produce: $20 \rightarrow 30 \rightarrow 40 \rightarrow 50 \rightarrow 60 \rightarrow 70 \rightarrow 80$. ¡Los datos quedan ordenados automáticamente!

Recorridos de Árboles Binarios

¿Qué es un recorrido?

Un **recorrido de árbol** es el proceso sistemático de **visitar todos los nodos** exactamente una vez, siguiendo un orden predefinido. La forma en que se visitan los nodos determina el resultado obtenido.

Inorden (LNR)

Izquierda → Nodo → Derecha

Preorden (NLR)

Nodo → Izquierda → Derecha

Postorden (LRN)

Izquierda → Derecha → Nodo

¿Para qué sirven?

→ Inorden

Obtener los elementos en **orden ascendente**. Ideal para ordenar datos.

→ Preorden

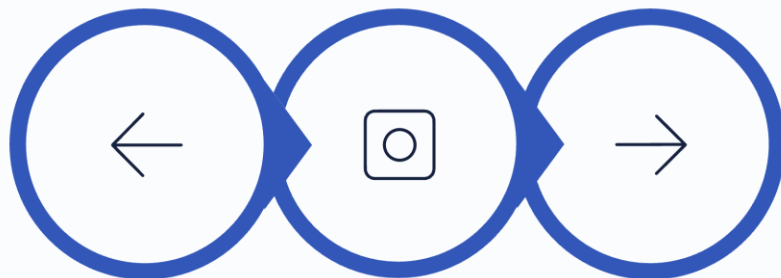
Copiar o serializar la estructura del árbol. Útil para almacenamiento.

→ Postorden

Eliminar el árbol de forma segura o evaluar expresiones matemáticas.

Recorrido Inorden (LNR)

El recorrido **Inorden** visita los nodos en el orden: **Izquierda** → **Nodo** → **Derecha**. Para un BST, este recorrido **siempre produce los elementos en orden ascendente**, lo que lo hace extremadamente útil.



Izquierda

Nodo

Derecha

Árbol ejemplo: Raíz=50, Izq=30 (hijos: 20, 40), Der=70 (hijos: 60, 80)

Resultado Inorden: 20 → 30 → 40 → 50 → 60 → 70 → 80 ✓

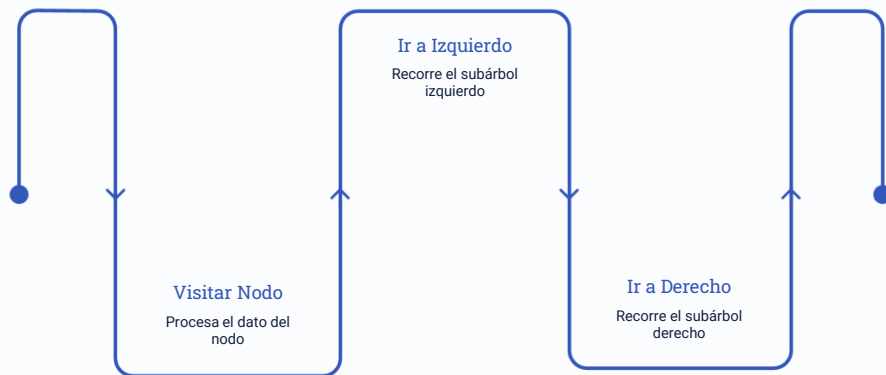
Implementación

```
void inorden(Nodo* nodo) {  
    if (nodo != nullptr) {  
        inorden(nodo->izquierdo);  
        cout << nodo->dato << " ";  
        inorden(nodo->derecho);  
    }  
}
```

✓ 💡 **Aplicación práctica:** Los sistemas de bases de datos usan Inorden para devolver resultados de consultas en orden alfabético o numérico sin necesidad de ordenarlos después.

Recorrido Preorden (NLR)

El recorrido **Preorden** sigue el orden: **Nodo** → **Izquierda** → **Derecha**. Visita la raíz **antes** de explorar cualquier subárbol. Es ideal para **copiar y reconstruir** la estructura exacta del árbol.



Árbol ejemplo: Raíz=50, Izq=30 (hijos: 20, 40), Der=70 (hijos: 60, 80)

Resultado Preorden: 50 → 30 → 20 → 40 → 70 → 60 → 80

Implementación

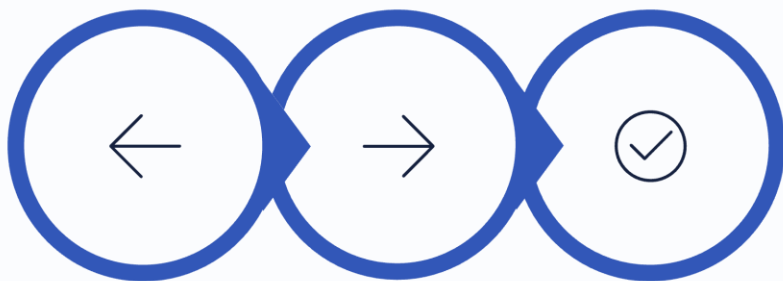
```
void preorden(Nodo* nodo) {  
    if (nodo != nullptr) {  
        cout << nodo->dato << " ";  
        preorden(nodo->izquierdo);  
        preorden(nodo->derecho);  
    }  
}
```

Aplicaciones Reales

- ▼ **Serialización de árboles:** guardar la estructura completa en un archivo.
- ▼ **Compiladores:** recorren el árbol sintáctico para generar código fuente.
- ▼ **Sistemas de archivos:** listar directorios antes que su contenido.

Recorrido Postorden (LRN)

El recorrido **Postorden** sigue el orden: **Izquierda** → **Derecha** → **Nodo**. El nodo actual se procesa **al final**, solo después de haber procesado ambos subárboles. Perfecto para operaciones de **liberación de memoria**.



Izquierda

Derecha

Visitar

Árbol ejemplo: Raíz=50, Izq=30 (hijos: 20, 40), Der=70 (hijos: 60, 80)

Resultado Postorden: 20 → 40 → 30 → 60 → 80 → 70 → 50

Implementación

```
void postorden(Nodo* nodo) {  
    if (nodo != nullptr) {  
        postorden(nodo->izquierdo);  
        postorden(nodo->derecho);  
        cout << nodo->dato << " ";  
    }  
}
```

Aplicaciones Reales

- ▼ **Eliminación segura:** borra los hijos antes que el padre para no perder referencias.
- ▼ **Evaluación de expresiones:** calcula operandos antes que el operador.
- ▼ **Garbage collection:** liberación de memoria en lenguajes como Java y C#.

Comparación de los Tres Recorridos

Usando el mismo árbol (Raíz=50, Izq=30, Der=70, con nodos 20, 40, 60, 80) los tres recorridos producen resultados completamente distintos. La elección del recorrido depende de la operación que se desea realizar.

Recorrido	Orden de Visita	Resultado (árbol ejemplo)	Caso de Uso Principal
Inorden	Izq → Nodo → Der	20, 30, 40, 50 , 60, 70, 80	Ordenar datos, consultas BD
Preorden	Nodo → Izq → Der	50 , 30, 20, 40, 70, 60, 80	Copiar árbol, serializar
Postorden	Izq → Der → Nodo	20, 40, 30, 60, 80, 70, 50	Eliminar árbol, evaluar expresiones

 **Patrón clave:** La diferencia entre los recorridos es **únicamente el momento** en que se procesa el nodo actual: antes (Pre), entre (In) o después (Post) de los subárboles.

¡Pon a Prueba tu Conocimiento!

Dado el BST con los valores **50 (raíz)**, **30**, **70**, **20**, **40**, **60**, **80** responde las siguientes preguntas en el chat o en la actividad colaborativa:

1

Recorrido Inorden

¿Cuál es la secuencia completa al hacer un recorrido Inorden? ¿Qué patrón observas en el resultado?

2

Inserción de Nuevo Nodo

Si insertas el valor **55**, ¿en qué posición exacta del árbol quedaría? Traza el camino nodo por nodo.

3

Eliminación Compleja

Si eliminas el nodo raíz **50** (que tiene dos hijos), ¿qué valor lo reemplazaría según la regla del sucesor Inorden?

Ejercicio Práctico: Construcción y Recorridos de un Árbol Binario de Búsqueda (BST)

Instrucciones

Se desea almacenar una serie de números utilizando un **Árbol Binario de Búsqueda (BST)**. Recuerde que en un BST:

- Los valores menores se insertan a la izquierda del nodo.
- Los valores mayores se insertan a la derecha del nodo.

Parte 1: Construcción del árbol

Inserte los siguientes valores en el orden indicado:

55, 35, 75, 25, 45, 65, 85, 15

Parte 2: Dibujo del árbol

Una vez finalizada la inserción:

1. Dibuje el Árbol Binario de Búsqueda resultante.
2. Muestre claramente la posición de cada nodo.

Parte 3: Recorridos

Realice los siguientes recorridos sobre el árbol construido y escriba la secuencia de valores obtenida en cada caso:

a) Recorrido Inorden (Izquierda → Raíz → Derecha)

Resultado: 15, 25, 35, 45, 55, 65, 75, 85

b) Recorrido Preorden (Raíz → Izquierda → Derecha)

Resultado: 55, 35, 25, 15, 45, 75, 65, 85

c) Recorrido Postorden (Izquierda → Derecha → Raíz)

Resultado: 15, 25, 45, 35, 65, 85, 75, 55

Implementación Completa en C++

Struct Nodo y Clase BST

```
#include
using namespace std;

struct Nodo {
    int dato;
    Nodo* izquierdo;
    Nodo* derecho;
    Nodo(int d) : dato(d),
        izquierdo(nullptr),
        derecho(nullptr) {}
};

class BST {
public:
    Nodo* raiz;
    BST() : raiz(nullptr) {}

    void insertar(int dato) {
        raiz = _insertar(raiz, dato);
    }

    Nodo* _insertar(Nodo* n, int d) {
        if (!n) return new Nodo(d);
        if (d < n->dato)
            n->izquierdo = _insertar(
                n->izquierdo, d);
        else if (d > n->dato)
            n->derecho = _insertar(
                n->derecho, d);
        return n;
    }
};
```

Búsqueda y Recorridos

```
bool buscar(Nodo* n, int d) {
    if (!n) return false;
    if (d == n->dato) return true;
    if (d < n->dato)
        return buscar(n->izquierdo, d);
    return buscar(n->derecho, d);
}

void inorden(Nodo* n) {
    if (n) {
        inorden(n->izquierdo);
        cout << n->dato << " ";
        inorden(n->derecho);
    }
}

// Uso:
int main() {
    BST arbol;
    int vals[] = {50,30,70,20,40,60,80};
    for (int v : vals)
        arbol.insertar(v);
    inorden(arbol.raiz);
    // Salida: 20 30 40 50 60 70 80
    return 0;
}
```


BST en el Mundo Real



Bases de Datos

Los índices B-Tree de PostgreSQL, MySQL y Oracle son extensiones de BST. Permiten búsquedas en millones de registros en microsegundos sin escanear toda la tabla.



Sistemas de Archivos

NTFS (Windows) y ext4 (Linux) usan variantes de árboles BST para organizar directorios y archivos, permitiendo acceso rápido independientemente del tamaño del disco.



Motores de Búsqueda

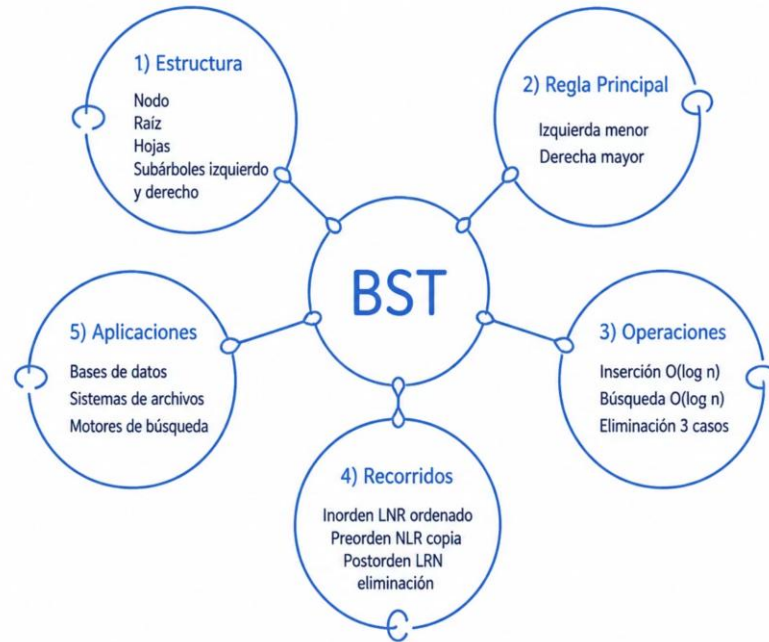
Los índices invertidos de Google y Bing utilizan estructuras de árbol para asociar palabras clave con documentos, haciendo posibles búsquedas en menos de un segundo.



Enrutamiento de Redes

Los routers usan árboles para almacenar tablas de enrutamiento IP, determinando el camino óptimo para cada paquete de datos de forma instantánea.

Mapa Conceptual: Todo lo que Aprendiste



🌳 Estructura

Jerárquica, con regla $izq < raíz < der$

⚡ Eficiencia

$O(\log n)$ en búsqueda e inserción

🔄 Recorridos

Inorden, Preorden, Postorden

🌐 Uso Real

BD, archivos, motores de búsqueda

Quiz: ¿Qué tanto aprendiste hoy?

Responde las siguientes 5 preguntas. El nivel de dificultad aumenta progresivamente. Usa el sistema de respuesta del aula virtual o escribe tus respuestas en el chat.

1

★ Básico

En un BST, los valores menores al nodo actual se almacenan en el subárbol: **A) Derecho B) Izquierdo C) Cualquiera**

2

★★ Fácil

¿Cuál es la complejidad temporal promedio de la búsqueda en un BST balanceado? **A) $O(n)$ B) $O(n^2)$ C) $O(\log n)$**

3

★★★ Medio

El recorrido Inorden de un BST produce los elementos en: **A) Orden descendente B) Orden aleatorio C) Orden ascendente**

4

★★★★ Difícil

Al eliminar un nodo con dos hijos, ¿qué valor lo reemplaza? **A) El mayor del subárbol izq B) El menor del subárbol der C) La raíz del árbol**

  **Respuestas:** 1-B · 2-C · 3-C · 4-B